

Papelaria SGC

Entrega 2 - Backend Funcional

ENTREGA 2

Backend Funcional

API REST completa em Django + DRF com JWT, arquitetura em camadas e testes automatizados

Disciplina:	Desenvolvimento Web
Projeto:	Sistema de Gestao Comercial - Papelaria
Equipe:	Pedro Henrique Bastos Guimaraes - 22303912
	Raphael Damiao Seabra Rosano - 22303595
Data:	Mai de 2026
Repositorio:	https://github.com/pedrobasgui/papelaria-sgc
Testes:	40 testes automatizados (todos passando)

1. Resumo da Entrega

Esta entrega contempla a **implementacao completa do backend** do sistema, contendo a API REST funcional, integracao com banco de dados, autenticao JWT, controle de acesso por perfil, validacao de regras de negocio e a interface web consumindo a API. Os criterios da Entrega 2 - Integracao com banco, Navegabilidade, Tratamento de excecoes, Organizacao do codigo e Versionamento - estao todos atendidos.

O que foi entregue

- **API REST completa** com todos os endpoints CRUD para Clientes, Produtos, Vendas e Relatorios.
- **Autenticacao JWT** (access + refresh) com controle de perfil (ADMIN / FUNCIONARIO).
- **Banco de dados integrado** (SQLite em dev, PostgreSQL em prod).
- **Registro de vendas** com transacao atomica: validacao de estoque, criacao da venda, debito do estoque.
- **Cancelamento de vendas** com devolucao automatica de estoque.
- **Relatorios**: por periodo, por cliente, anual (grafico) e produtos mais vendidos.
- **Interface web** em Django + Bootstrap 5 + Chart.js consumindo a API REST via Fetch.
- **40 testes automatizados** (pytest-django) cobrindo regras de negocio criticas.
- **Tratamento global de excecoes** com handler customizado.
- **Documentacao Swagger/OpenAPI** automatica em /api/docs/.

2. Estrutura do Projeto

O código segue a arquitetura em camadas definida na Entrega 1, com cada módulo de negócio implementado como um Django App independente:

```
papelaria-sgc/
├── manage.py                # Entry-point do Django
├── requirements.txt
├── pytest.ini               # Configuracao do pytest
├── README.md
├──
├── papelaria_sgc/           # Configuracao
│   ├── settings.py          # Settings (DEV/PROD)
│   ├── urls.py              # Rotas raiz
│   └── wsgi.py
│   └──
│   ├── apps/
│   │   ├── core/            # Nucleo
│   │   │   ├── exceptions.py # DomainException + filhas
│   │   │   ├── handlers.py   # Exception handler global
│   │   │   ├── permissions.py # IsAdmin, IsAdminOrFuncionario
│   │   │   └── management/commands/popular_dados.py
│   │   └──
│   │   ├── usuarios/        # AUTH + JWT
│   │   │   ├── models.py     # Usuario customizado + perfil
│   │   │   ├── serializers.py
│   │   │   ├── views.py      # Login, refresh, me, etc.
│   │   │   └── urls.py
│   │   └──
│   │   ├── clientes/        # CRUD com arquitetura em camadas
│   │   │   ├── models.py     # Cliente + validar_cpf
│   │   │   ├── repositories.py # ClienteRepository
│   │   │   ├── services.py    # ClienteService
│   │   │   ├── serializers.py
│   │   │   ├── views.py
│   │   │   └── urls.py
│   │   └──
│   │   ├── produtos/        # idem
│   │   │   └── vendas/       # Venda + ItemVenda
│   │   │       └── services.py # criar_venda atomica
│   │   └──
│   │   ├── relatorios/      # Strategy pattern
│   │   │   └── strategies.py # 4 estrategias de relatorio
│   │   └──
│   │   └── web/              # Views Django + templates
│   └──
│   ├── templates/           # HTML
│   ├── static/              # CSS + JS (cliente da API)
│   ├── database/script.sql   # Script PostgreSQL
│   ├── docs/                # PDFs e diagramas
│   └── tests/                # 40 testes pytest
```

3. Endpoints da API REST

Todos os endpoints retornam **JSON**. Excetuando login e recuperacao de senha, todos exigem o header Authorization: Bearer <token>.

Metodo	URL	Descricao	Auth
POST	/api/auth/login/	Login (retorna access + refresh)	publica
POST	/api/auth/refresh/	Renova access token	refresh
POST	/api/auth/logout/	Blacklist do refresh	JWT
GET	/api/auth/me/	Dados do usuario logado	JWT
POST	/api/auth/trocar-senha/	Troca de senha	JWT
POST	/api/auth/recuperar-senha/	Stub p/ Entrega 3	publica
GET	/api/clientes/?busca=...	Listar clientes	JWT
POST	/api/clientes/	Criar cliente	JWT
GET	/api/clientes/{id}/	Detalhe	JWT
PUT	/api/clientes/{id}/	Atualizar	JWT
DELETE	/api/clientes/{id}/	Excluir (bloqueado se houver vendas)	JWT
GET	/api/produtos/?busca=...	Listar produtos	JWT
GET	/api/produtos/estoque-baixo/	Produtos abaixo do minimo	JWT
POST	/api/produtos/	Criar produto	ADMIN
PUT	/api/produtos/{id}/	Atualizar	ADMIN
DELETE	/api/produtos/{id}/	Remover/inativar	ADMIN
GET	/api/vendas/	Listar vendas (com filtros)	JWT
GET	/api/vendas/{id}/	Detalhe da venda	JWT
POST	/api/vendas/	Registrar venda (atomica)	JWT
POST	/api/vendas/{id}/cancelar/	Cancelar e devolver estoque	JWT
POST	/api/vendas/por-periodo/	Vendas em periodo	JWT
GET	/api/vendas/por-cliente/{id}/	Vendas do cliente	JWT
POST	/api/relatorios/por-periodo/	Relatorio por periodo	ADMIN
GET	/api/relatorios/por-cliente/{id}/	Historico do cliente	ADMIN
GET	/api/relatorios/anual/?ano=YYYY	Grafico anual (12 meses)	ADMIN
GET	/api/relatorios/top-produtos/	Top N produtos vendidos	ADMIN

4. Exemplos de Uso da API

4.1 Login

Request:

POST /api/auth/login/ { "username": "admin", "password": "admin123" }

Response 200:

```
{
  "access": "eyJhbGciOiJIUzI1NiIs... ",
  "refresh": "eyJhbGciOiJIUzI1NiIs... ",
  "usuario": {
    "id": 1,
    "username": "admin",
    "email": "admin@mundoletrado.com.br",
    "perfil": "ADMIN"
  }
}
```

4.2 Criar venda

Request:

POST /api/vendas/ (Authorization: Bearer ...)

```
{
  "cliente_id": 1,
  "itens": [
    { "produto_id": 1, "quantidade": 2 },
    { "produto_id": 2, "quantidade": 5 }
  ]
}
```

Response 201 (resumido):

```
{
  "id": 1,
  "data": "2026-05-06T18:30:00",
  "cliente": { "id": 1, "nome": "Ana Beatriz Souza", "cpf": "529.982.247-25" },
  "usuario": { "id": 1, "username": "admin", "perfil": "ADMIN" },
  "valor_total": "62.30",
  "status": "CONCLUIDA",
  "itens": [ ... ]
}
```

4.3 Erro padronizado (CPF duplicado)

```
{
  "erro": "Ja existe um cliente com este CPF.",
  "codigo": "CPF_DUPLICADO",
  "detalhes": { "cpf": "529.982.247-25" }
}
```

Status HTTP: **409 Conflict.** O handler global `apps.core.handlers.custom_exception_handler` captura todas as `DomainException` e padroniza o formato.

5. Tratamento de Excecoes

O tratamento de excecoes segue uma hierarquia de classes e e centralizado num handler global, conforme criterio da entrega.

Hierarquia

```
DomainException (apps/core/exceptions.py)
■■■ CPFDuplicadoError           [409, CPF_DUPLICADO]
■■■ EmailInvalidoError         [400, EMAIL_INVALIDO]
■■■ ClientePossuiVendasError   [409, CLIENTE_COM_VENDAS]
■■■ PrecoInvalidoError         [400, PRECO_INVALIDO]
■■■ EstoqueInsuficienteError   [409, ESTOQUE_INSUFICIENTE]
■■■ VendaSemItensError         [400, VENDA_SEM_ITENS]
■■■ VendaJaCanceladaError      [409, VENDA_JA_CANCELADA]
■■■ PermissaoNegadaError       [403, PERMISSAO_NEGADA]
```

Handler Global

Configurado em `settings.REST_FRAMEWORK`:

```
REST_FRAMEWORK = {
    "EXCEPTION_HANDLER": "apps.core.handlers.custom_exception_handler",
    ...
}
```

Toda `DomainException` e convertida em JSON com status, codigo e detalhes; demais excecoes do DRF (autenticacao, validacao) sao envelopadas no mesmo formato. Isso garante **respostas consistentes** em toda a API.

6. Testes Automatizados

Foram implementados **40 testes automatizados** com pytest-django, cobrindo modelos, services e a camada de API.

Arquivo	Casos	O que testa
test_auth.py	5	JWT login, credencial invalida, rotas protegidas, /me
test_clientes.py	13	validacao CPF, CRUD, CPF duplicado (409), CPF invalido
test_produtos.py	11	tem_estoque, debito/credito, preco negativo, permissoes
test_vendas.py	11	criar venda atomica, estoque insuficiente, cancelamento
TOTAL	40	Todos passando ✓

Como rodar os testes

```
$ pytest tests/ -v

===== test session starts =====
collected 39 items

tests/test_auth.py ::TestAuth ..... PASSED
tests/test_clientes.py ::TestValidar... PASSED
tests/test_clientes.py ::TestClienteService PASSED
tests/test_clientes.py ::TestClienteAPI PASSED
tests/test_produtos.py ::TestProdutoModel PASSED
tests/test_produtos.py ::TestProdutoService PASSED
tests/test_produtos.py ::TestProdutoAPI PASSED
tests/test_vendas.py ::TestVendaService PASSED
tests/test_vendas.py ::TestVendaAPI PASSED

===== 39 passed in 13.77s =====
```

7. Navegabilidade das Telas (Web)

A interface web foi construída em Django Templates + Bootstrap 5 + Chart.js. Todas as telas consomem a API REST via Fetch JavaScript (arquivo `static/js/api.js`) usando o token JWT armazenado no `localStorage`.

■ ■ /login/	(pública)
■ ■ /dashboard/	(KPIs + gráfico anual)
■ ■ /clientes/	(listar + busca)
■ ■ ■ /clientes/novo/	(form criar)
■ ■ ■ /clientes/{id}/editar/	(form editar)
■ ■ /produtos/	(listar + busca)
■ ■ ■ /produtos/novo/	(ADMIN)
■ ■ ■ /produtos/{id}/editar/	(ADMIN)
■ ■ /vendas/	(listar + filtros)
■ ■ ■ /vendas/nova/	(PDV simplificado)
■ ■ ■ /vendas/{id}/	(detalhe + cancelar)
■ ■ /relatorios/	(3 abas: periodo, cliente, anual)

Funcionalidades de UX

- Token JWT armazenado em `localStorage` com refresh transparente.
- Toasts para feedback de operações (sucesso/erro).
- Confirmações em exclusões e cancelamentos.
- Máscara automática de CPF e validação em tempo real.
- Filtros e busca com debounce.
- Gráficos interativos (Chart.js) com tooltip em português.
- Layout responsivo (Bootstrap 5).
- Cores e fontes consistentes (identidade visual definida).

8. Como Executar

```
# 1. Clonar
git clone https://github.com/pedrobasgui/papelaria-sgc.git
cd papelaria-sgc

# 2. Ambiente virtual
python3 -m venv .venv
source .venv/bin/activate

# 3. Dependencias
pip install -r requirements.txt

# 4. Banco de dados (cria as tabelas)
python manage.py migrate

# 5. Popula dados iniciais
python manage.py popular_dados

# 6. Sobe o servidor
python manage.py runserver

# Acesse: http://localhost:8000/
# Login: admin / admin123
```

Endpoints uteis apos rodar

http://localhost:8000/	Tela de login
http://localhost:8000/dashboard/	Dashboard com KPIs
http://localhost:8000/api/docs/	Swagger UI (interativo)
http://localhost:8000/api/schema/	Schema OpenAPI (JSON)
http://localhost:8000/admin/	Django Admin

Fim do documento - Entrega 2